

Quasidifferential cryptanalysis

From [Differential Cryptanalysis in the Fixed-Key Model](#)

Traditional Differential

$$F : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m (\Delta_{in} = a, \Delta_{out} = b)$$

$$F(x) + F(x + a) = b$$

Differential Distribution Table (DDT), each entry is:

$$DDT_{a,b}^F = |\{x \in \mathbb{F}_2^n \mid F(x) + F(x + a) = b\}| = 2^n \Pr[F(x) + F(x + a) = b]$$

Assumption: **x is uniform at random**, $p \gg 2^{-n}$, so the data required to attack is $O(1/p)$.

Therefore, the adversary treats **the independence holds for each round**.

$$\begin{aligned} & \Pr[F(x) + F(x + a_1) = a_{r+1}] \\ &= \sum_{a_2, \dots, a_r} \Pr[\Lambda_{i=1}^r F_i(x_i) + F_i(x_i + a_i) = a_{i+1}] \\ &= \prod_{i=1}^r \Pr[F_i(z_i) + F_i(z_i + a_i) = a_{i+1}] \end{aligned}$$

Evaluate it with DDT (The probability with the strong independence assumption). **with clustering**

$$DDT_{a_1, a_{r+1}}^F / 2^n = \sum_{a_2, \dots, a_r} \prod_{i=1}^r DDT_{a_i, a_{i+1}}^{F_i} / 2^n$$

The counterexample where the assumption fails:

$$F = F_2 \circ F_1, \text{ where } F_2 = F_1^{-1}.$$

Linear Attack

For mask $\Lambda_{in} = u, \Lambda_{out} = v$, calculate the correlation of a function F :

$$C_{v,u}^F = 2 \cdot \Pr_x[v^\top F(x) + u^\top x = 0] - 1$$

To depict the propagation of linear correlation, there are two functions:

$$\delta_x(y) = \begin{cases} 1, & x = y \\ 0, & x \neq y \end{cases}$$

which is the **standard orthonormal basis** of the inner product $\langle f, g \rangle = \sum_{x \in \mathbb{F}_2^n} f(x)g(x)$.

for example: $\delta_x(y) = [0, 0, 0, 1], [0, 0, 1, 0], \dots$

The **group characteristics** form a set of bases, $\chi_u(x) = (-1)^{u^T x}$, which makes binary field addition and non-zero multiplication groups a homomorphism.

That is $\chi_u(x + t) = \chi_u(x)\chi_u(t)$.

Specially, $\langle \chi_u, \chi_v \rangle = 2^n \delta_u(v)$.

Definition. Fourier Transformation

$$f: \mathbb{F}_2^n \rightarrow \mathbb{R}$$

$$(\mathcal{F}_n f)(u) = \langle \chi_u, f \rangle = \sum_{x \in \mathbb{F}_2^n} (-1)^{u^T x} f(x)$$

map the standard basis $(\delta_x(y))$ to the characteristic basis $\chi_u(x)$.

for function $g(x) = f(x + t)$ or $y = g(x) = x + t$, we have:

$$\begin{aligned} (\mathcal{F}_n g)(u) &= \langle \chi_u, g \rangle \\ &= \langle \chi_u(x), f(x + t) \rangle. \text{ let } (y = x + t), \text{ since } x \in \mathbb{F}_2^n, \text{ then } y \in \mathbb{F}_2^n \text{ (shift only).} \\ &= \langle \chi_u(y + t), f(y) \rangle = \sum_{y \in \mathbb{F}_2^n} \chi_u(y + t) f(y) \\ &= \chi_u(t) \sum_{y \in \mathbb{F}_2^n} \chi_u(y) f(y). \text{ homomorphism of } \chi_u. \\ &= \chi_u(t) \langle \chi_u(y), f(y) \rangle \\ &= \chi_u(t) (\mathcal{F}_n g)(u) \end{aligned}$$

Definition. Transition Matrix

Map the input x to the output $y = F(x)$.

- ROW = $F(x)$, COLUMN = x .
- If F is bijective, T^F is a permutation Matrix: (* one "1" in each row/col; *orthogonal Matrix)

Example: $n = 2, \mathbb{F}_{2^2} = \{00, 01, 10, 11\}, F(00) = 01, F(01) = 00, F(10) = 11, F(11) = 10$,

$$T^F = \begin{pmatrix} & x = 00 & x = 01 & x = 10 & x = 11 \\ y = F(x) = 00 & 0 & 0 & 1 & 0 \\ y = F(x) = 01 & 1 & 0 & 0 & 0 \\ y = F(x) = 10 & 0 & 0 & 0 & 1 \\ y = F(x) = 11 & 0 & 1 & 0 & 0 \end{pmatrix}$$

So the function is described as a matrix: $\vec{y} = T^F \vec{x}$.

Definition. Correlation Matrix

$F: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$. correlation matrix $C^F: \mathbb{R}[\mathbb{F}_2^n] \rightarrow \mathbb{R}[\mathbb{F}_2^m], C^F = \mathcal{F}_m T^F \mathcal{F}_n^{-1}$.

Construct A Fourier Matrix (FM):

- A Fourier Matrix is a **Walsh-Hadamard matrix, symmetric and orthogonal**. $F^{-1} = 1/nF$.
- ROW index = mask, COL index = value
- Entry in $\mathcal{F}_M$ is $\mathcal{F}[u, x] = \chi_u(x) = (-1)^{u^T x}$

Example:

$n = 2, x, u \in \{00, 01, 10, 11\}$

- $u = 00, \chi_{00}(x) \rightarrow [(-1)^0, (-1)^0, (-1)^0, (-1)^0] = [1, 1, 1, 1]$.
- $u = 01, \chi_{01}(x) \rightarrow [(-1)^{0 \cdot 0 \oplus 1 \cdot 0}, (-1)^{0 \cdot 0 \oplus 1 \cdot 1}, (-1)^{0 \cdot 1 \oplus 1 \cdot 0}, (-1)^{0 \cdot 1 \oplus 1 \cdot 1}] = [1, -1, 1, -1]$.

.....

Fourier Matrix $\mathcal{F}_M$:

	$F(x) = 00$	$F(x) = 01$	$F(x) = 10$	$F(x) = 11$
$v = 00$	1	1	1	1
$v = 01$	1	-1	1	-1
$v = 10$	1	1	-1	-1
$v = 11$	1	-1	-1	1

$$\mathcal{F}_M^{-1} = 1/4 \mathcal{F}^T:$$

$$1/4 \times \begin{pmatrix} & v=00 & v=01 & v=10 & v=11 \\ x=00 & 1 & 1 & 1 & 1 \\ x=01 & 1 & -1 & 1 & -1 \\ x=10 & 1 & 1 & -1 & -1 \\ x=11 & 1 & -1 & -1 & 1 \end{pmatrix}$$

$C^F = \mathcal{F}_m T^F \mathcal{F}_n^{-1}$ is the correlation matrix; the entry in C^F is:

$$C_{v,u}^F = 2P_r[v^T F(x) + u^T x = 0] - 1$$

WHY ??

$C^F = \mathcal{F}_m T^F \mathcal{F}_n^{-1}$, the entry

$$C_{v,u}^F = 1/2^m \cdot \langle \chi_v, T^F \chi_u \rangle, \text{ entry in } \mathcal{F} \text{ is } \chi$$

$$= 1/2^m$$

- $T^F \mathcal{F}_n^{-1} = \sum_{x \in \mathbb{F}_2^n} T^F[y, x] \mathcal{F}_n^{-1}[x, v] = T^F \mathcal{F}_n^{-1}[y, v]$, $\mathcal{F}^{-1}$ maps the output y to the out-mask v .
- $\mathcal{F}_m(T^F \mathcal{F}_n^{-1}) = \sum_{y \in \mathbb{F}_2^m} \mathcal{F}[u, y](T^F \mathcal{F}_n^{-1}[y, v]) = \mathcal{F}_m T^F \mathcal{F}_n^{-1}[u, v]$, $\mathcal{F}$ maps the input x to the in-mask u .

Means, map the value space of $(x$ and $y)$ to the mask space $(u$ and $v)$

Finally, the entries of the result matrix represent: $\sum_{x \in \mathbb{F}_2^n} (-1)^{ux+vy}$.

Quasi

Quasidifferential Basis

For differential attacks, the input for each function is a **pair**, such as (x_0, x_1) , satisfying $x_0 + x_1 = a$. We need to map (x_0, x_1) to the mask; then it will be convenient to use the Fourier transform.

Definition. Quasidifferential basis

Let u be the mask, a be the difference. $\beta_{u,a}(x, y) : \mathbb{F}_2^n \times \mathbb{F}_2^n \rightarrow \mathbb{R}$.

fixed

$$\beta_{u,a}(x_0, x_1) = (\chi_u(x_0) \delta_a(x_0 + x_1))$$

where:

- $\chi_u(x_0)$ is to map the value to the mask (the same for x_1 since $x_1 = x_0 + a$ and a is fixed);
- $\delta_a(x_0 + x_1)$ is to filter out only the value that satisfies the difference.

Theorem. properties of $\beta_{u,a}$

- **Shift invariance:**

$$\beta_{u,a}(x + t, y + t) = \chi_u(t) \beta_{u,a}(x, y)$$

$$\begin{aligned} & \beta_{u,a}(x + t, y + t) \\ &= \chi_u(x + t) \delta_a(x + y + t + t) \\ &= \chi_u(t) \cdot \chi_u(x) \delta_a(x + y), \text{ since } \chi_u \text{ homomorphism} \\ &= \chi_u(t) \beta_{u,a}(x, y) \end{aligned}$$

- **Orthogonality:**

$$\langle \beta_{v,b}, \beta_{u,a} \rangle = 2^n \delta_v(u) \delta_b(a).$$

- $a \neq b, x + y = a = b$ is impossible, $\langle \beta_{v,b}, \beta_{u,a} \rangle = 0$;

◦ $a = b$,

$$\begin{aligned}
& \langle \beta_{v,b}, \beta_{u,a} \rangle \\
&= \sum_{(x,y) \in \mathbb{F}_2^n \times \mathbb{F}_2^n} \chi_v(x) \delta_a(x+y) \chi_u(x) \delta_a(x+y) \\
&= \sum_{(x,y) \in \mathbb{F}_2^n \times \mathbb{F}_2^n} \chi_v(x) \chi_u(x) \delta_a(x+y), \text{ where } (u = x = v) \text{ and } (x+y = a = b) \\
&= \sum_{(x,y) \in \mathbb{F}_2^n \times \mathbb{F}_2^n} \chi_v(u) \delta_a(b) \\
&= 2^n \delta_v(u) \delta_b(a).
\end{aligned}$$

By analogy with the Fourier transform (replacing the function from χ_u to $\beta_{u,a}$), we obtain the quasi-difference transform operator:

$$\mathcal{Q}_n : \mathbb{R}[\mathbb{F}_2^n \times \mathbb{F}_2^n] \rightarrow [\mathbb{F}_2^n \times \mathbb{F}_2^n] \text{ using } (\mathcal{Q}_n f)(u, a) = \langle \beta_{u,a}, f \rangle$$

$(\mathcal{Q}_n f)(u, a)$ represents: The coefficient of the linear mask u when the difference is fixed at a .

$(\mathcal{Q}_n f)(u, a)/2^n$ is the coordinate.

Quasidifferential Transition Matrix

- Linear Attack: T^F maps x to $F(x)$.
- Differential Attack: $T^F \otimes T^F$ maps (x_0, x_1) to $F(x_0), F(x_1) = y_0, y_1$.

$$(T^F \otimes T^F)_{(y_1, y_2), (x_1, x_2)} = T_{y_1, x_1}^F T_{y_2, x_2}^F = \delta_{y_1}(F(x_1)) \delta_{y_2}(F(x_2)).$$

suppose $x, y \in \mathbb{F}_2^2$.

The size of $(T^F \otimes T^F)_{(y_1, y_2), (x_1, x_2)}$ is 4×4 .

And the inside entry = 1 iff. $y_1 = F(x_1)$ and $y_2 = F(x_2)$.

$$X = \begin{pmatrix} x_{11} & x_{12} \\ x_{21} & x_{22} \end{pmatrix}, \quad Y = \begin{pmatrix} y_{11} & y_{12} \\ y_{21} & y_{22} \end{pmatrix}. \quad X \otimes Y = \begin{pmatrix} x_{11}Y & x_{12}Y \\ x_{21}Y & x_{22}Y \end{pmatrix} = \begin{pmatrix} x_{11}y_{11} & x_{11}y_{12} & x_{12}y_{11} & x_{12}y_{12} \\ x_{11}y_{21} & x_{11}y_{22} & x_{12}y_{21} & x_{12}y_{22} \\ x_{21}y_{11} & x_{21}y_{12} & x_{22}y_{11} & x_{22}y_{12} \\ x_{21}y_{21} & x_{21}y_{22} & x_{22}y_{21} & x_{22}y_{22} \end{pmatrix}.$$

Definition. Quasidifferential Transition Matrix

n, m denote the size of input and output, $F : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$. Under the quasidifferential basis $T^F \otimes T^F$, Quasidifferential Transition Matrix D^F is defined as

$$D^F = \mathcal{Q}_m(T^F \otimes T^F) \mathcal{Q}_n^{-1}$$

Means to map the input and output differences onto the mask $(a, b) \rightarrow (u, v)$. And $\mathcal{Q}_n^{-1} = 1/n \mathcal{Q}_n^T$.

Each entry in D^F is (ROW = (v, b) , COL = (u, a)):

$$D_{(v,b),(u,a)}^F = \langle \delta_{(v,b)}, \mathcal{Q}_n(T^F \otimes T^F) \mathcal{Q}_n^T \delta_{(u,a)} \rangle / 2^n = \langle \beta_{v,b}, (T^F \otimes T^F) \beta_{u,a} \rangle / 2^n.$$

Methods:

- For a matrix M , the entry at position (i, j) is $M_{i,j} = \langle \delta_i, M \delta_j \rangle$, where δ_i and δ_j are both unit column vectors, and they are 1 only at the position (i, j) .

$$D_{(v,b),(u,a)}^F = \langle \delta_{(v,b)}, D_{(v,b),(u,a)}^F \delta_{(u,a)} \rangle$$

Using $\delta_{v,b}$ and $\delta_{u,a}$ to locate the corresponding points in the matrix $D_{(v,b),(u,a)}^F$.

- Transpose transfer: $\langle u, Av \rangle = \langle A^T u, v \rangle$.
- $D_{(v,b),(u,a)}^F = \begin{pmatrix} D_{(0,0),(0,0)} & D_{(0,0),(0,1)} & D_{(0,0),(1,0)} & D_{(0,0),(1,1)} \\ D_{(0,1),(0,0)} & D_{(0,1),(0,1)} & D_{(0,1),(1,0)} & D_{(0,1),(1,1)} \\ D_{(1,0),(0,0)} & D_{(1,0),(0,1)} & D_{(1,0),(1,0)} & D_{(1,0),(1,1)} \\ D_{(1,1),(0,0)} & D_{(1,1),(0,1)} & D_{(1,1),(1,0)} & D_{(1,1),(1,1)} \end{pmatrix}.$

- $\beta_{u,a}(x, y) = \chi_u(x)\delta_a(x + y)$, where $\mathcal{Q}_n$ is generated by $\chi_u(x)$ (ROW= u , COL= x). Therefore, when mapped to a certain point, $\mathcal{Q}_n \cdot \delta = \beta$ (δ is used to locate, and $\mathcal{Q}$ is used to map $value \rightarrow correlation$).

Calculation of D^F :

$$\begin{aligned}
D_{(v,b),(u,a)}^F &= \langle \beta_{v,b}, (T^F \otimes T^F) \beta_{u,a} \rangle / 2^n, \quad (T^F \otimes T^F) \beta_{u,a}(x_1, x_2) = \beta_{u,a}(y_1, y_2) \\
&= 1/2^n \cdot \langle \beta_{v,b}(x_1, x_2), \beta_{u,a}(y_1, y_2) \rangle, \quad (\text{swap } x \text{ and } y, x \text{ and } y \text{ are dual and can be swapped.}) \\
&= 1/2^n \cdot \langle \beta_{v,b}(y_1, y_2), \beta_{u,a}(x_1, x_2) \rangle \\
&= 1/2^n \cdot \sum_{x_1, x_2 \in \mathbb{F}_2^n \times \mathbb{F}_2^n} \beta_{v,b}(y_1, y_2) \cdot \beta_{u,a}(x_1, x_2) \\
&= 1/2^n \cdot \sum_{x_1, x_2 \in \mathbb{F}_2^n \times \mathbb{F}_2^n} \chi_u(x_1) \delta_a(x_1 + x_2) \cdot \chi_v(y_1) \delta_b(y_1 + y_2) \\
&= 1/2^n \cdot \sum_{x \in \mathbb{F}_2^n, y = F(x) \text{ and } x+a=F(x)+b} (-1)^{u^T x + v^T y}
\end{aligned}$$

```

for ui in (mask u)
  for ai in (diff a)
    for xi in (value x)
      calculate DF

```

% from the property of the Fourier matrix, this process is allowed to speed up.

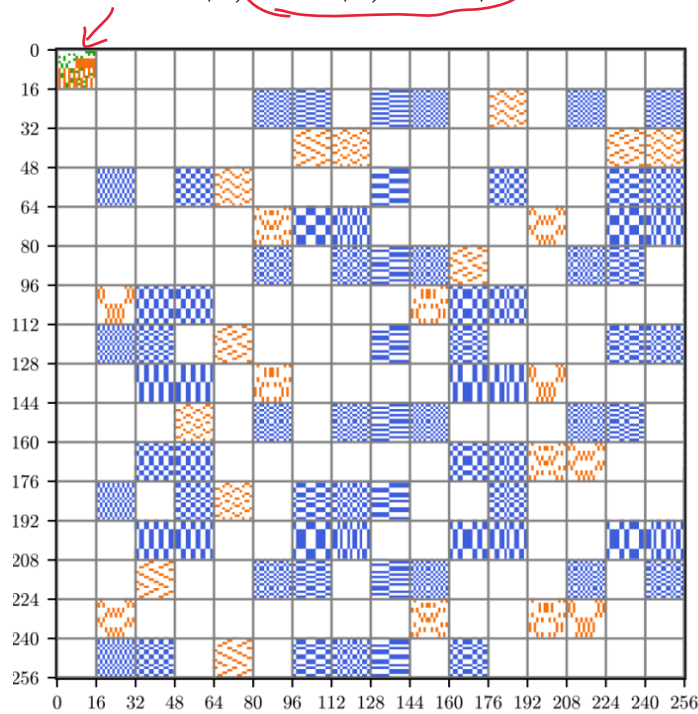
D^F for the 4-bit sbox of RECTANGLE:

ROW = (u, a), $2^4 \times 2^4 = 256$.

COL ...

DDT

绿 : 1/2, 黄 : 1/4, 蓝 : 1/8



Theorem. properties of D^F .

For $D_{(v,b),(u,a)}^F = \frac{1}{2^n} \cdot \sum_{x_1 \in \mathbb{F}_2^n, F(x+a)=F(x)+b} (-1)^{u^T x + v^T y}$. The classical differential property and linear correlation are:

$$\bullet \quad u = v = \vec{0}, D_{(0,b),(0,a)}^F = \frac{\sum_{x \in \mathbb{F}_2^n, x+a=F(x)+b} 1}{2^n} = P_r(a \rightarrow b) = \frac{DDT_{a,b}^F}{2^{-n}}.$$

- $a = b = \vec{0}, D_{(v,0),(u,0)}^F = \frac{\sum_{x \in \mathbb{F}_2^n} (-1)^{u^T x + v^T y}}{2^n} = C_{u,v}^F$.

Properties:

- (1) If F is a bijection, then D^F is an orthogonal matrix.
- (2) If $F = (F_1, \dots, F_m)$, then $D^F = \bigotimes_{i=1}^m D^{F_i}$. (boxed maps)
- (3) If $F = F_2 \circ F_1$, then $D^F = D^{F_2} D^{F_1}$. (composition)
- (4) If $F(x) = x + t$ for some $t \in \mathbb{F}_2^m$, then $D_{(v,b),(u,a)}^F = \chi_v(t) \delta_v(u) \delta_b(a)$.
- (5) If F is a linear function, then $D_{(v,b),(u,a)}^F = \delta_u(F^T(v)) \delta_b(F(a))$.

1. If F is bijective, then D^F is an orthogonal matrix.

F is bijective $\implies T^F$ is a permutation matrix (orthogonal) && $\mathcal{Q}$ is an orthogonal matrix.

- the property of the tensor product: $(AB) \otimes (CD) = (A \otimes C)(B \otimes D)$

2.

- The transition matrix can be decomposed as:

$$T_{y,x}^F = \delta_y(F(x)) = \prod_{i=1}^m \delta_{y_i}(F_i(x_i)) = \prod_{i=1}^m T_{y_i,x_i}^{F_i} = \left(\bigotimes_{i=1}^m T^{F_i} \right)_{y,x}.$$

$$\text{so } T^F \otimes T^F = \left(\bigotimes_{i=1}^m T^{F_i} \right) \otimes \left(\bigotimes_{i=1}^m T^{F_i} \right) = \bigotimes_{i=1}^m (T^{F_i} \otimes T^{F_i}).$$

- Decompose $\mathcal{Q}$:

$$\beta_{(u_1, \dots, u_m), (a_1, \dots, a_m)}((x_1, \dots, x_m), (y_1, \dots, y_m)) = \prod_{i=1}^m \chi_{u_i}(x_i) \delta_{a_i}(x_i + y_i) = \prod_{i=1}^m \beta_{u_i, a_i}(x_i, y_i) = \bigotimes_{i=1}^m \mathcal{Q}_{m_i}.$$

- So, D^F :

$$\begin{aligned} D^F &= \mathcal{Q}_m (T^F \otimes T^F) \mathcal{Q}_n^{-1} \\ &= \left(\bigotimes_{i=1}^m \mathcal{Q}_{m_i} \right) \left(\bigotimes_{i=1}^m (T^{F_i} \otimes T^{F_i}) \right) \left(\bigotimes_{i=1}^m \mathcal{Q}_{n_i}^{-1} \right). \end{aligned}$$

$$D^F = \bigotimes_{i=1}^m (\mathcal{Q}_{m_i} (T^{F_i} \otimes T^{F_i}) \mathcal{Q}_{n_i}^{-1}) = \bigotimes_{i=1}^m D^{F_i}.$$

3.

$$(T^{F_2} T^{F_1}) \otimes (T^{F_2} T^{F_1}) = (T^{F_2} \otimes T^{F_2})(T^{F_1} \otimes T^{F_1}).$$

$$\begin{aligned} D^F &= \mathcal{Q}_m (T^{F_2} \otimes T^{F_2}) (\mathcal{Q}_p^{-1} \mathcal{Q}_p) (T^{F_1} \otimes T^{F_1}) \mathcal{Q}_n^{-1} \\ &= [\mathcal{Q}_m (T^{F_2} \otimes T^{F_2}) \mathcal{Q}_p^{-1}] [\mathcal{Q}_p (T^{F_1} \otimes T^{F_1}) \mathcal{Q}_n^{-1}] \\ &= D^{F_2} D^{F_1}. \end{aligned}$$

4.

$$F(x + a) = (x + a) + t = x + a + t, \text{ and } F(x) + b = (x + t) + b = x + t + b$$

$$\text{Condition: } F(x + a) = F(x) + b, \text{ so } x + a + t = x + t + b \implies a = b$$

$$\begin{aligned} D^F &= 1/2^n \cdot \sum_{x \in \mathbb{F}_2^n, y=F(x) \text{ and } x+a=F(x)+b} (-1)^{u^T x + v^T y} \\ &\quad (\text{where } (-1)^{u^T x + v^T F(x)} = (-1)^{u^T x + v^T (x+t)} = (-1)^{u^T x + v^T x + v^T t} = (-1)^{(u+v)^T x} \cdot (-1)^{v^T t}) \\ &= 1/2^n \cdot \sum_{x \in \mathbb{F}_2^n, y=F(x) \text{ and } x+a=F(x)+b} (-1)^{(u+v)^T x} \cdot (-1)^{v^T t} \\ &\quad \left(\sum_x (-1)^{(u+v)^T x} = 1 \text{ iff } u = v; \text{ otherwise } = 0 \right) \\ &= \chi_v(t) \delta_b(a) \delta_v(u) \end{aligned}$$

5.

$$\text{Let } F(x) = Mx, \text{ so}$$

$$F(x + a) = Mx + Ma$$

$$\text{Condition: } F(x + a) = F(x) + b = Mx + b$$

$$\text{we get } Ma = b \text{ (} F(a) = b \text{)}$$

$$D^F = 1/2^n \cdot \sum_{x \in \mathbb{F}_2^n, y=F(x) \text{ and } x+a=F(x)+b} (-1)^{u^T x + v^T (Mx)}$$

$$(where (-1)^{u^T x + v^T (Mx)} = (-1)^{(u+M^T v)^T x}, M(v) = F(v).$$

$$(iff. (u + M^T v) = 0, D^F = 2^n; otherwise 0)$$

$$= \delta_u(F^\top(v)) \delta_b(F(a))$$

Quasidifferential Trail

Definition. Quasidifferential Trail

Quasidifferential Trail is a sequence $\omega_1, \dots, \omega_{r+1}$, where $\omega_i = (u_i, a_i)$ is the **mask-differential pair**.

The **correlation** of this quasidifferential is $\prod_{i=1}^r D_{\omega_{i+1}, \omega_i}^F$.

- if all inner masks are $u_i = 0$, $\prod_{i=1}^r D_{(0, a_{i+1}), (0, a_i)}^F = \prod_{i=1}^r P_r[F_i(x + a_i) = F_i(x) + a_{i+1}]$.

$$D_{\omega_{r+1}, \omega_1}^{\mathcal{F}} = \sum_{\omega_2, \dots, \omega_r} \prod_{i=1}^r D_{\omega_{i+1}, \omega_i}^{\mathcal{F}_i}.$$

$\prod_{i=1}^r D_{\omega_{i+1}, \omega_i}^{\mathcal{F}_i}$ is from **property (3)** above, and where $\sum_{\omega_2, \dots, \omega_r}$ corresponds to the **clustering effect**.

The clustering on the mask

$$\Pr \left[\bigwedge_{i=1}^r F_i(x_i + a_i) = F_i(x_i) + a_{i+1} \right] = \sum_{u_2, \dots, u_r} \prod_{i=1}^r D_{(u_{i+1}, a_{i+1})}^{(F_i, (u_i, a_i))},$$

For the differential $a_i \rightarrow a_{i+1} \rightarrow \dots \rightarrow a_r$, there are **many masks** that support this propagation.

The **Active Sbox** in Quasidiff trail: the associated **differential/mask != 0**.

==> means in the propagation, non-zero masks will activate more sbox, thereby **decreasing** the overall correlation.

==> $D_{(v,b), (u,a)}^F = 1/2^n \cdot \sum_{x \in \mathbb{F}_2^n, y=F(x) \text{ and } x+a=F(x)+b} (-1)^{u^T x + v^T y}$, the **max** value is when $u = v = 0$.

Theorem. properties of Quasidifferential trail

Quasidifferential trail: $F = F_r \circ \dots \circ F_1$, characteristic $a_1, \dots, a_{r+1}$ with correlation p . (p is the max correlation)

- (1) If $(u_1, a_1), \dots, (u_{r+1}, a_{r+1})$ is a quasidifferential trail with correlation $(-1)^b p$ where $b \in \{0, 1\}$, then for any quasidifferential trail $(v_1, a_1), \dots, (v_{r+1}, a_{r+1})$ with correlation c , the correlation of the quasidifferential trail $(u_1 + v_1, a_1), \dots, (v_{r+1} + u_{r+1}, a_{r+1})$ is $(-1)^b c$.
- (2) If the correlations of any number of quasidifferential trails with differences $a_1, \dots, a_{r+1}$ and correlation $\pm p$ sum to zero, then the probability of the characteristic $a_1, \dots, a_{r+1}$ is zero.

(1)

correlation is p : $D_{(u=0, a_{i+1}), (v=0, a_i)}^F = 1/2^n \cdot \sum_{x \in \mathbb{F}_2^n, y=F(x) \text{ and } x+a_i=F(x)+a_{i+1}} (-1)^{u_i^T x + u_{i+1}^T F(x)} = p(-1)^b$

- $(-1)^b$ corresponds to the **cancellation (+1 and -1)**. ==> $u_i^T x + u_{i+1}^T F(x) = b_i$

so

$$\begin{aligned}
D_{(u_{i+1}+v_{i+1}, a_{i+1}), (u_i+v_i, a_i)}^F &= 1/2^n \cdot \sum_{x \in \mathbb{F}_2^n, y=F(x) \text{ and } x+a_i=F(x)+a_{i+1}} (-1)^{(u_i+v_i)^T x + (u_{i+1}+v_{i+1})^T F(x)} \\
&= 1/2^n \cdot \sum_{x \in \mathbb{F}_2^n, y=F(x) \text{ and } x+a_i=F(x)+a_{i+1}} (-1)^{(u_i^T x + u_{i+1}^T F(x)) + (v_i^T x + v_{i+1}^T F(x))} \\
&= (-1)^b \cdot 1/2^n \cdot \sum_{x \in \mathbb{F}_2^n, y=F(x) \text{ and } x+a_i=F(x)+a_{i+1}} (-1)^{v_i^T x + v_{i+1}^T F(x)} \\
&= (-1)^b \cdot c
\end{aligned}$$

(2)

Each correlation can be canceled by another one.

Add Key

let $F(x) = G(x) + k$

- $F(x + a) = G(x + a) + k = F(x) + b = G(x) + k + b$, i.e., $G(x + a) = G(x) + b$.

- $D^F = 1/2^n \cdot \sum_{x \in \mathbb{F}_2^n, y=F(x) \text{ and } x+a=F(x)+b} (-1)^{u_i^T x + u_{i+1}^T F(x)}$, where
 $u_i^T x + v_i^T F(x) = u_i^T x + u_{i+1}^T G(x) + u_{i+1}^T k_i$.

$$D^F = (-1)^{u_{i+1}^T k_i} \cdot 1/2^n \cdot \sum_{x \in \mathbb{F}_2^n, y=F(x) \text{ and } x+a=F(x)+b} (-1)^{u_i^T x + u_{i+1}^T G(x)}$$

$$\text{we have: } D_{\omega_{r+1}, \omega_1}^{\mathcal{F}} = \sum_{\omega_2, \dots, \omega_r} \prod_{i=1}^r D_{\omega_{i+1}, \omega_i}^{\mathcal{F}_i}$$

$$\text{so } D_{\omega_{r+1}, \omega_1}^{\mathcal{F}} = \sum_{\omega_2, \dots, \omega_r} \prod_{i=1}^r (-1)^{u_{i+1}^T k_i} D_{\omega_{i+1}, \omega_i}^{\mathcal{G}_i},$$

Theorem. Quasidifferential trail on keyed cipher.

Theorem 4.3. *Let $F = F_r \circ \dots \circ F_1$ with $F_i(x) = G_i(x) + k_i$. If $k = (k_1, \dots, k_r)$ is a uniform random variable on a set $\mathcal{K}$, then*

$$\Pr[F(x + a) = F(x) + b] = \sum_{\substack{u_2, \dots, u_r \\ a_2, \dots, a_r \\ (u_2, \dots, u_r) \perp \mathcal{K}}} \prod_{i=1}^r D_{(u_{i+1}, a_{i+1}), (u_i, a_i)}^{\mathcal{G}_i},$$

where $u_1 = u_{r+1} = 0$ and the probability is over a uniform random x and over the keys $k_1, \dots, k_r$. In particular, for $\mathcal{K} = (\mathbb{F}_2^n)^r$, only quasidifferential trails with zero masks contribute to the key-averaged probability of the differential.

That is, when the **key is uniform and random**, the classical differential attack is accurate for probability evaluation.

In other words, the **key brings enough randomness** to the cipher.

To compute the Quasidifferential Transition Matrix (QTM)

Q-DDT (QTM) searching for Sbox

The basic method of QTM generation.

```

for ui in (mask u)
  for ai in (diff a)
  {
    for vi in (mask v)
      for bi in (diff b)
        check if x satisfies "F(x+a) = F(x) + b"
  }
calculate DF

```

Basic method: The complexity of calculating QTM is $O(2^{2n+2m} \cdot 2^n) = O(2^{3n+2m})$,

For example, $n = m = 4$ (sbox of RECTANGLE), the complexity is $2^{3 \cdot 4 + 2 \cdot 4} = 2^{20}$.

Faster method:

- interleave coding;
- Walsh-Hadamard Matrix generation.

The complexity is $O(2^{2n+2m} \cdot (m + n))$

Constraints for And/Add-RX

- bitwise and: $\wedge$, $and(x||y)$.
- bitwise or: $\vee$.
- bitwise complement: $\bar{x}$
- integer modulo and: $add(x||y)$.
- $x \preceq y$: for i in $1..to..n$ $x_i \leq y_i$.

Bitwise-And:

$$D_{(w,c),(u||v,a||b)}^{and} = 2^{-wt(a \vee b) - wt(w \wedge \bar{a} \wedge \bar{b})} (-1)^{(\bar{a} \wedge u + c \wedge v)^T (\bar{b} \wedge v + c \wedge u) + (u \wedge v)^T (c + a \wedge b \wedge \bar{c})}.$$

"And" acts on each bit independently. so $G^{and_n} = (G^{and_1})^{\otimes n}$

$$D_{(w,c),(u||v,a||b)}^{and_n} = \prod_{i=1}^n D_{(w_i,c_i),(u_i||v_i,a_i||b_i)}^{and_1}$$

For 1 bit, we have $G^F = 1/2^n \cdot \sum_{x \in \mathbb{F}_2^n, y=F(x) \text{ and } x+a=F(x)+b} (-1)^{u^T x + v^T y}$.

$$D_{(w,c),(u||v,a||b)}^{(and_1)} = \frac{1}{2^{n=2}} \sum_{\substack{(x,y) \in \mathbb{F}_2^2 \\ (x+a)(y+b)=xy+c}} (-1)^{ux+vy+wxy}$$

- $2^{-wt(a \vee b)}$.

for any combination of (a, b, c) , $2^{2 \cdot 3}$, count how many (x, y) satisfy $(x + a)(y + b) = xy + c$

a	b	c	#(x,y)	condition
0	0	0	4	0=0
0	0	1	0	0=1
0	1	0	2	x=0, y ∈ {0,1}
.....				

$$= 2^{-(a \vee b)}. \text{ And } \prod 1/2^i = 2^{-(\sum_{bit} (a \vee b))}$$

- $\text{wt}(w \wedge \bar{a} \wedge \bar{b})$

$$\text{for any } u, v, x, y, \sum_{\substack{(x,y) \in \mathbb{F}_2^2 \\ (x+a)(y+b)=xy+c}} (-1)^{ux+vy} = (1 + (-1)) = 0.$$

xy	w=0, $(-1)^{wxy} =$	w=1
0 [0,0], [0,1], [1,0]	1	1
1 [1,1]	1	-1

so when $w \wedge \bar{a} \wedge \bar{b}$, there is a **reduction** of the correlation.

==> the following components can be explained in a similar way.

Bitwise-Add

CCZ equivalence, function add and q share the same Diff/Linear resistance.

- $D^{add} \rightarrow D^q = (D^M \circ D^{and})$

$$D_{(w,c),(u\|v,a\|b)}^{add} = D_{(w',c'),(u'\|v',a'\|b')}^q, \text{ where}$$

$$(a', b', c') = (b + c, a + c, M^\dagger(a + b + c)) \quad \text{and} \quad (u', v', w') = (u + w, v + w, M^T(u + v + w)).$$

where $M(x)_1 = 0$, since no carry on the first position, $M(x)_i = \sum_{j=1}^{i-1} x_j$, $M^\dagger(x) = [x + (x \ll 1)] \gg 1$.

M corresponds to **carry**.

- **map add to and:** $D_{(w,c),(u\|v,a\|b)}^{and} = \sum_{w',c',u',v',a',b'} D_{(w',c'),(u'\|v',a'\|b')}^{and}$

- $D = 2^{-\text{wt}(a' \vee b') - \text{wt}(w' \wedge \bar{a}' \wedge \bar{b}')} (-1)^s,$

$$\text{where } s = (\bar{a}' \wedge u' + c' \wedge v')^\top (\bar{b}' \wedge v' + c' \wedge u') + (u' \wedge v')^\top (c' + a' \wedge b' \wedge \bar{c}')$$

Code

code for brute force enumeration:

```
import numpy as np

SB = [0x6, 0x5, 0xC, 0xA, 0x1, 0xE, 0x7, 0x9, 0xB, 0x0, 0x3, 0xD, 0x8, 0xF, 0x4, 0x2]
n = 4

def bit_count(x):
    return bin(x).count('1')

def dot_product(u, x):
    return bit_count(u & x) % 2

def get_diff_pairs(a, b, n=4):
    size = 1 << n
    return [x for x in range(size) if SB[x ^ a] == SB[x] ^ b]

def calculate_matrix_element_bruteforce(a, b, u, v, n=4):
    total = 0
    diff_pairs = get_diff_pairs(a, b, n)

    for x in diff_pairs:
        fx = SB[x]
        sign = 1 if (dot_product(u, x) ^ dot_product(v, fx)) == 0 else -1
        total += sign

    return total / (1 << n)

def calculate_full_matrix_bruteforce(n=4):
    idx_size = 1 << (2 * n)
    d_matrix = np.zeros((idx_size, idx_size), dtype=np.float64)

    for u in range(1 << n):
        for a in range(1 << n):
            col_idx = (a << n) | u # 改为 a||u
            for v in range(1 << n):
                for b in range(1 << n):
                    row_idx = (b << n) | v # 改为 b||v
                    value = calculate_matrix_element_bruteforce(a, b, u, v, n)
                    d_matrix[row_idx, col_idx] = value

    return d_matrix

def save_matrix(matrix, filename="QMT.txt"):
    np.savetxt(filename, matrix, fmt='%g')
    print(f"saved as {filename}")

# print(calculate_matrix_element_bruteforce(2, 8, 1, 1))
QTM = calculate_full_matrix_bruteforce()
# print(QTM)
# np.savetxt("QMT.txt", QTM, fmt='%g', delimiter=' ')
```

code using Walsh Hadamard and...

```
import itertools
import time

def interleave_bits(x, y, n): #interleave code, mix x and y into one integer
    (x=x2x1,y=y2y1,z=y2x2y1x1)
    z = 0
    for i in range(n):
        z |= (x & (1 << i)) << i | (y & (1 << i)) << (i + 1);
    return z
```

```

def to_quasidifferential_basis(x):
    # walsh Hadamard Transition, 2*2: WH * [a0,a1]^T = [a0+a1, a0-a1]
    # Here is WH for interleave code.
    if len(x) == 1:
        return x

    assert len(x) % 4 == 0

    l = int(len(x) / 4)

    x_00 = to_quasidifferential_basis(x[0:l])
    x_01 = to_quasidifferential_basis(x[l:2*l])
    x_10 = to_quasidifferential_basis(x[2*l:3*l])
    x_11 = to_quasidifferential_basis(x[3*l:4*l])

    return vector(x.base_ring(),
        (x_00 + x_11).list() + \
        (x_01 + x_10).list() + \
        (x_00 - x_11).list() + \
        (x_01 - x_10).list())

def interleaved_transition_matrix(F, n, m):
    # generate matrix 2^2m row (x index) and 2^2n col (F(x) index) using interleave code
    T = matrix(QQ, 2**(2*m), 2**(2*n))
    for x in range(2**n):
        for y in range(2**n):
            i = interleave_bits(x, y, n)
            j = interleave_bits(F(x), F(y), m)
            T[j, i] = 1
    return T

def quasidifferential_transition_matrix(F, n, m):
    D = interleaved_transition_matrix(F, n, m)

    # Transform columns
    for i in range(2**(2*n)):
        D.set_column(i, to_quasidifferential_basis(D.column(i)))
    # Transform rows
    for i in range(2**(2*m)):
        D.set_row(i, to_quasidifferential_basis(D.row(i)))

    return D / 2**n

def deinterleave_quasidifferential_transition_matrix(D, n, m, primary = 'diff'):
    R = matrix(QQ, 2**(2*m), 2**(2*n))
    for u, v in itertools.product(range(2**n), range(2**m)):
        for a, b in itertools.product(range(2**n), range(2**m)):
            if primary == 'mask':
                R[2**m*v + b, 2**n*u + a] = D[
                    interleave_bits(b, v, m), interleave_bits(a, u, n)
                ]
            elif primary == 'diff':
                R[2**m*b + v, 2**n*a + u] = D[
                    interleave_bits(b, v, m), interleave_bits(a, u, n)
                ]
    return R

# Example:
from sage.crypto.sboxes import Rectangle
D = quasidifferential_transition_matrix(Rectangle, 4, 4)
D = deinterleave_quasidifferential_transition_matrix(D, 4, 4)

```

